

# Core OOP Concepts

**Prof. Ashwani Mishra**

Assistant Professor  
AIML Department

## Content

|                                          |    |
|------------------------------------------|----|
| 1. Introduction to OOP.....              | 3  |
| 2. Classes, Objects & Constructors.....  | 8  |
| 3. Encapsulation & Access Modifiers..... | 17 |
| 4. Inheritance & Polymorphism.....       | 22 |
| 5. Abstract Classes & Interfaces.....    | 32 |
| 6.                                       |    |
| References.....                          | 41 |

## What is Object-Oriented Programming?

- OOP organizes software as a collection of objects that interact by sending messages.
- An object combines data (attributes) and behavior (methods) in a single unit.
- Classes act as blueprints; objects are concrete instances created from those blueprints.
- OOP models real-world entities such as students, accounts, and shapes in code.
- The four pillars are encapsulation, inheritance, polymorphism, and abstraction.
- Java is a pure OOP language where every line of code lives inside a class.

## Figure 2.1 Object-Oriented Programming

### Four Pillars of OOP

| Pillar        | Meaning                          | Java Mechanism                          |
|---------------|----------------------------------|-----------------------------------------|
| Encapsulation | Hide data, expose methods        | private fields + public getters/setters |
| Inheritance   | Reuse code from parent class     | extends keyword, super()                |
| Polymorphism  | One interface, many forms        | Overloading and overriding              |
| Abstraction   | Hide complexity, show essentials | abstract class, interface               |

Figure 2.1 Object-Oriented Programming

# Procedural vs Object-Oriented

## Procedural vs Object-Oriented

| Procedural Programming Language                             | Object Oriented Programming Language                         |
|-------------------------------------------------------------|--------------------------------------------------------------|
| 1. Program is divided into functions.                       | 1. Program is divide into classes and objects..              |
| 2. The emphasis is on doing things.                         | 2. The emphasis on data.                                     |
| 3. Poor modeling to real world problems.                    | 3. Strong modeling to real world problems.                   |
| 4. It is not easy to maintain project if it is too complex. | 4. It is easy to maintain project even if it is too complex. |
| 5. Provides poor data security.                             | 5. Provides strong data Security.                            |
| 6. It is not extensible programming language.               | 6. It is highly extensible programming language.             |
| 7. Productivity is low.                                     | 7. Productivity is high.                                     |
| 8. Do not provide any support for new data types.           | 8. Provide support to new Data types.                        |
| 9. Unit of programming is function.                         | 9. Unit of programming is class.                             |
| 10. Ex. Pascal , C , Basic , Fortran.                       | 10. Ex. C++ , Java , Oracle.                                 |

Figure 2.2 Procedural vs OOP

## **Benefits of OOP in Java**

- **Modularity:** each class handles one responsibility, simplifying testing and maintenance.
- **Reusability:** inheritance and interfaces let you reuse existing code in new contexts.
- **Flexibility:** polymorphism allows replacing components without changing client code.
- **Security:** encapsulation hides internal state and exposes only controlled public methods.
- **Scalability:** large systems (banking, ERP) are built from cooperating objects and packages.
- **Industry demand:** OOP skills with Java are essential for enterprise and Android development.

## Class vs Object

| Class                                                        | Object                                                        |
|--------------------------------------------------------------|---------------------------------------------------------------|
| 1) Class is a collection of similar objects                  | 1) Object is an instance of a class                           |
| 2) Class is conceptual (is a template)                       | 2) Object is real                                             |
| 3) No memory is allocated for a class.                       | 3) Each object has its own memory                             |
| 4) Class can exist without any objects                       | 4) Objects can't exist without a class                        |
| 5) Class does not have any values associated with the fields | 5) Every object has its own values associated with the fields |

Figure 2.4 Class vs Object

## Defining a Class

- A class declares fields (data) and methods (behavior) that describe a type of object.
- Fields hold the state of each object; methods define operations on that state.
- The public access modifier on a class allows it to be used from other packages.
- Instance variables get default values; explicit initialization can happen in constructors.
- Methods use void when they perform actions without returning a value.
- Good class design uses meaningful names and keeps related data and behavior together.



## Figure 2.5 Class Definition

Student.java

```
public class Student {  
    // Instance variables (fields)  
    private int rollNo;  
    private String name;  
    private double marks;  
  
    // Method to display student info  
    public void display() {  
        System.out.println(rollNo + " " + name + " " + marks);  
    }  
}
```

Figure 2.5 Class Definition

## Creating Objects

- The new keyword allocates memory on the heap and calls the class constructor.
- A reference variable (Student s1) stores the address of the object, not the object itself.
- Multiple references can point to the same object; two new calls create two distinct objects.
- When no reference points to an object, the garbage collector eventually frees its memory.
- Printing an object with println shows the class name @ hashCode unless toString is overridden.
- Object creation is the foundation for all OOP programs in Java.

## Figure 2.6 Object Creation

### CreateObjects.java

```
public class CreateObjects {  
    public static void main(String[] args) {  
        // Create objects using new keyword  
        Student s1 = new Student();  
        Student s2 = new Student();  
  
        // s1 and s2 are separate objects in heap memory  
        System.out.println("s1: " + s1);  
        System.out.println("s2: " + s2);  
    }  
}
```

Figure 2.6 Object Creation

## Constructors

- A constructor is a special method that runs automatically when an object is created with new.
- Constructors have the same name as the class and no return type (not even void).
- The no-arg constructor sets safe defaults; parameterized constructors accept initial values.
- If you define any constructor, Java does not supply a default no-arg constructor.
- Constructor chaining calls another constructor in the same class using `this(...)` first.
- Always initialize all fields in constructors to avoid undefined object state.

## Figure 2.7 Constructor Example

StudentConstructors.java

```
public class Student {  
    private int rollNo;  
    private String name;  
  
    // Default constructor (no-arg)  
    public Student() {  
        this.rollNo = 0;  
        this.name = "Unknown";  
    }  
  
    // Parameterized constructor  
    public Student(int rollNo, String name) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
}
```

Figure 2.7 Constructor Example

## The this Keyword

- this refers to the current object instance inside an instance method or constructor.
- Use this.field when a parameter name shadows an instance field name.
- this() calls another constructor in the same class and must be the first statement.
- Returning this from a method enables method chaining (builder pattern).
- this cannot be used in static methods because static methods belong to the class, not an object.
- Understanding this clarifies how Java resolves names between local and instance scope.

## Figure 2.8 this Keyword

**ThisKeyword.java**

```
public class Student {  
    private String name;  
  
    public Student(String name) {  
        this.name = name; // this.name = instance field  
    }  
  
    public void setName(String name) {  
        this.name = name; // disambiguate parameter vs field  
    }  
  
    public Student getReference() {  
        return this; // return current object  
    }  
}
```

Figure 2.8 this Keyword

## Encapsulation Explained

- Encapsulation bundles data and the methods that operate on that data within a class.
- Internal fields are declared private so outside code cannot access them directly.
- Public getter and setter methods provide controlled read and write access to private data.
- Validation logic in setters prevents invalid states (negative balance, empty name).
- Encapsulation hides implementation details and exposes only a stable public interface.
- Well-encapsulated classes are easier to modify internally without breaking client code.



## Figure 2.9 Encapsulation

### EncapsulationDemo.java

```
public class BankAccount {  
    private double balance; // hidden from outside  
  
    public BankAccount(double initial) {  
        if (initial >= 0) balance = initial;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance)  
            balance -= amount;  
    }  
}
```

Figure 2.9 Encapsulation

# Access Modifiers

## Java Access Modifiers

| Modifier  | Class | Package | Subclass | World |
|-----------|-------|---------|----------|-------|
| public    | Yes   | Yes     | Yes      | Yes   |
| protected | Yes   | Yes     | Yes      | No    |
| default   | Yes   | Yes     | No       | No    |
| private   | Yes   | No      | No       | No    |

Figure 2.10 Access Modifiers

## Encapsulation with Getters and Setters

- The balance field is private, so external code must use deposit and withdraw methods.
- getBalance returns the current balance without allowing direct modification.
- deposit checks that amount is positive before adding to balance.
- withdraw ensures the amount is positive and does not exceed the available balance.
- This pattern protects object integrity and centralizes business rules in one place.
- IDEs can generate getters and setters, but you should add validation where needed.

## Figure 2.11 Encapsulation Example

### EncapsulationDemo.java

```
public class BankAccount {  
    private double balance; // hidden from outside  
  
    public BankAccount(double initial) {  
        if (initial >= 0) balance = initial;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance)  
            balance -= amount;  
    }  
}
```

Figure 2.11 Encapsulation Example

# Inhe



Figure 2.12 Inheritance Hierarchy

## Implementing Inheritance

- The extends keyword declares that Student is a subclass of Person.
- The subclass inherits non-private fields and methods from the parent (superclass).
- super(name) in the Student constructor calls the Person constructor to initialize name.
- protected fields like name are accessible in subclasses but not from unrelated classes.
- Java supports single inheritance for classes: a class extends at most one direct parent.
- All classes implicitly extend java.lang.Object if no extends clause is written.

## Figure 2.13 Inheritance Example

InheritanceDemo.java

```
class Person {  
    protected String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
  
    public void display() {  
        System.out.println("Person: " + name);  
    }  
}  
  
class Student extends Person {  
    private int rollNo;  
  
    public Student(String name, int rollNo) {  
        super(name); // call parent constructor  
        this.rollNo = rollNo;  
    }  
  
    @Override  
    public void display() {  
        System.out.println("Student: " + name + ", Roll " + rollNo);  
    }  
}
```

Figure 2.13 Inheritance Example

## Method Overloading

- Overloading defines multiple methods with the same name but different parameter lists.
- The compiler selects the correct overload based on argument types and count at compile time.
- Changing only the return type does not create a valid overload in Java.
- Overloading improves readability: one intuitive name (add) for related operations.
- Constructors are commonly overloaded to support different object creation scenarios.
- Overloading is compile-time (static) polymorphism because the binding happens at compile time.



## Figure 2.14 Method Overloading

### MethodOverloading.java

```
public class Calculator {  
    // Same name, different parameters  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public double add(double a, double b) {  
        return a + b;  
    }  
  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Figure 2.14 Method Overloading

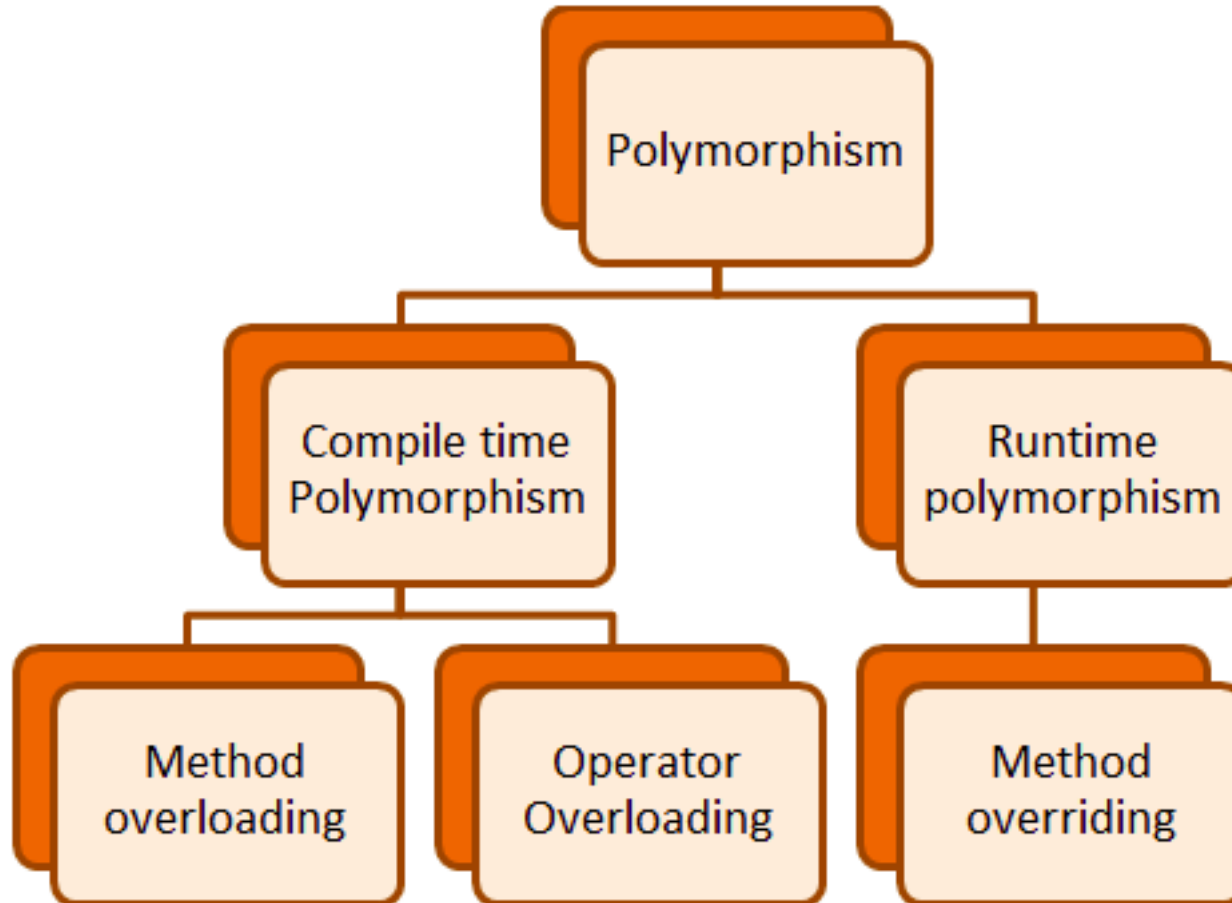
# Overloading vs Overriding

## Overloading vs Overriding

| Method Overloading                                                                          | Method Overriding                                                                               |
|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Methods must have the same name but different parameters or different number of parameters. | Methods must have the same signature (same name and same parameters)                            |
| Method overloading is a compile-time polymorphism                                           | Method overriding is a run-time polymorphism                                                    |
| Method overloading may or may not require inheritance. It occurs within the same class.     | Method overriding always needs inheritance. It occurs between super class and sub class         |
| When an error occurs, it can be caught at compile time                                      | When an error occurs, it can be caught at run time                                              |
| Method overloading enhances code readability                                                | Provide the specific implementation of the method that is already provided by the parent class. |
| Return type can be same or different. But you must have to change the parameter.            | Return type must be same or covariant                                                           |
| Static binding is used                                                                      | Dynamic binding is used                                                                         |
| Overloading is applicable to both private and final methods.                                | Overriding is not applicable for private and final methods.                                     |
| The argument list should be different                                                       | The argument list should be the same                                                            |

## Polymorphism in Java

- Polymorphism means one interface can represent different underlying forms (many shapes).
- Runtime polymorphism occurs when a parent reference points to a child object.
- The JVM calls the overridden method of the actual object type, not the reference type.
- This dynamic method dispatch enables flexible designs with arrays of mixed subclasses.
- Method overriding requires the same method signature and compatible return type.
- Use `@Override` to catch signature mistakes at compile time when redefining parent methods.



## Figure 2.16 Runtime Polymorphism

### Runtime Polymorphism

|   | Compile-Time Polymorphism                                                                | Runtime Polymorphism                                                                |
|---|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1 | This is <b>resolved by the compiler</b> .                                                | This is <b>not resolved by the compiler</b> .                                       |
| 2 | This is also known as <b>static/early binding</b> or <b>overloading</b> .                | This is also known as <b>dynamic/late binding</b> or <b>overriding</b> .            |
| 3 | The method name is same with <b>different parameters and return type</b> .               | The method name is same with the <b>same parameters and same return type</b> .      |
| 4 | Provides <b>fast execution</b> since the method to be executed is known at compile-time. | Provides <b>slow execution</b> since the method to be executed is known at runtime. |
| 5 | <b>Less flexible</b> since all things execute at compile-time.                           | <b>More flexible</b> since all things execute at runtime.                           |

Figure 2.16 Runtime Polymorphism

## Polymorphism Code Example

- Person p1 holds a Student object; Person p2 holds a Teacher object.
- Both variables are declared as Person but refer to different subclass instances.
- Calling p1.display() executes Student.display() because of dynamic binding.
- Calling p2.display() executes Teacher.display() at runtime.
- This technique supports extensibility: add new subclasses without changing existing client code.
- Polymorphism is central to frameworks, collections, and GUI event handling in Java.



## Figure 2.17 Polymorphism Example

### PolymorphismDemo.java

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
        Person p1 = new Student("Amit", 101);  
        Person p2 = new Teacher("Dr. Smith", "Java");  
  
        p1.display(); // calls Student.display()  
        p2.display(); // calls Teacher.display()  
  
        // Parent reference, child object - runtime binding  
    }  
}
```

Figure 2.17 Polymorphism Example

## Abstract Classes

- An abstract class cannot be instantiated directly; it serves as a base for subclasses.
- Abstract methods declare a signature without a body; subclasses must implement them.
- Abstract classes can contain both abstract and concrete (fully implemented) methods.
- Concrete fields and constructors in abstract classes are shared by all subclasses.
- Use abstract classes when related types share common state and partial implementation.
- Shape defines area() abstractly; Circle provides the specific formula for calculation.



## Figure 2.18 Abstract Class

### AbstractShape.java

```
abstract class Shape {  
    protected String color;  
  
    public Shape(String color) {  
        this.color = color;  
    }  
  
    // Abstract method - no body in base class  
    public abstract double area();  
  
    public void showColor() {  
        System.out.println("Color: " + color);  
    }  
}  
  
class Circle extends Shape {  
    private double radius;  
  
    public Circle(String color, double radius) {  
        super(color);  
        this.radius = radius;  
    }  
  
    @Override  
    public double area() {  
        return Math.PI * radius * radius;  
    }  
}
```

Figure 2.18 Abstract Class

## Abstract Class vs Interface

Abstract Class vs Interface

| Abstract class                                                                                    | Interface                                                                                                          |
|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1. The <b>abstract keyword</b> is used to declare abstract class.                                 | The <b>interface keyword</b> is used to declare interface.                                                         |
| 2. Abstract class can have <b>abstract and non-abstract</b> methods.                              | Interface can have <b>only abstract</b> methods. Since Java 8, it can have <b>default and static methods</b> also. |
| 3. Abstract class <b>doesn't support multiple inheritance</b> .                                   | Interface <b>supports multiple inheritance</b> .                                                                   |
| 4. An <b>abstract class</b> can extend another Java class and implement multiple Java interfaces. | An <b>interface</b> can extend another Java interface only.                                                        |
| 5. Abstract class can have <b>final, non-final, static and non-static variables</b> .             | Interface has <b>only static and final variables</b> .                                                             |
| 6. An <b>abstract class</b> can be extended using keyword "extends".                              | An <b>interface</b> can be implemented using keyword "implements."                                                 |
| 7. A Java <b>abstract class</b> can have class members like private, protected, etc.              | Members of a Java interface are public by default.                                                                 |

Figure 2.19 Abstract Class vs Interface

## Implementing an Interface

- Drawable declares draw() and getColor() that implementing classes must define.
- Rectangle implements Drawable and provides concrete bodies for both methods.
- A class can implement multiple interfaces: class X implements A, B, C.
- Interface references can hold any implementing object, enabling polymorphic behavior.
- Constants in interfaces are public static final by default.
- Interfaces decouple what an object can do from how it is implemented internally.

## Figure 2.20 Interface Implementation

### Drawable.java

```
interface Drawable {  
    void draw();           // implicitly public abstract  
    String getColor();     // contract method  
}  
  
class Rectangle implements Drawable {  
    private String color;  
  
    public Rectangle(String color) {  
        this.color = color;  
    }  
  
    @Override  
    public void draw() {  
        System.out.println("Drawing rectangle");  
    }  
  
    @Override  
    public String getColor() {  
        return color;  
    }  
}
```

Figure 2.20 Interface Implementation

## Java Packages

- Packages group related classes and prevent naming conflicts across projects.
- Package names follow reverse domain convention: `com.parul.oopj` for university code.
- The package statement must be the first line in a Java source file.
- import statements bring classes from other packages into the current file scope.
- Directory structure must mirror package names when compiling with `javac -d`.
- Packages organize large OOP projects into modules such as `model`, `service`, and `util`.

**Figure 2.21 Package Organization**

## Packages in Java

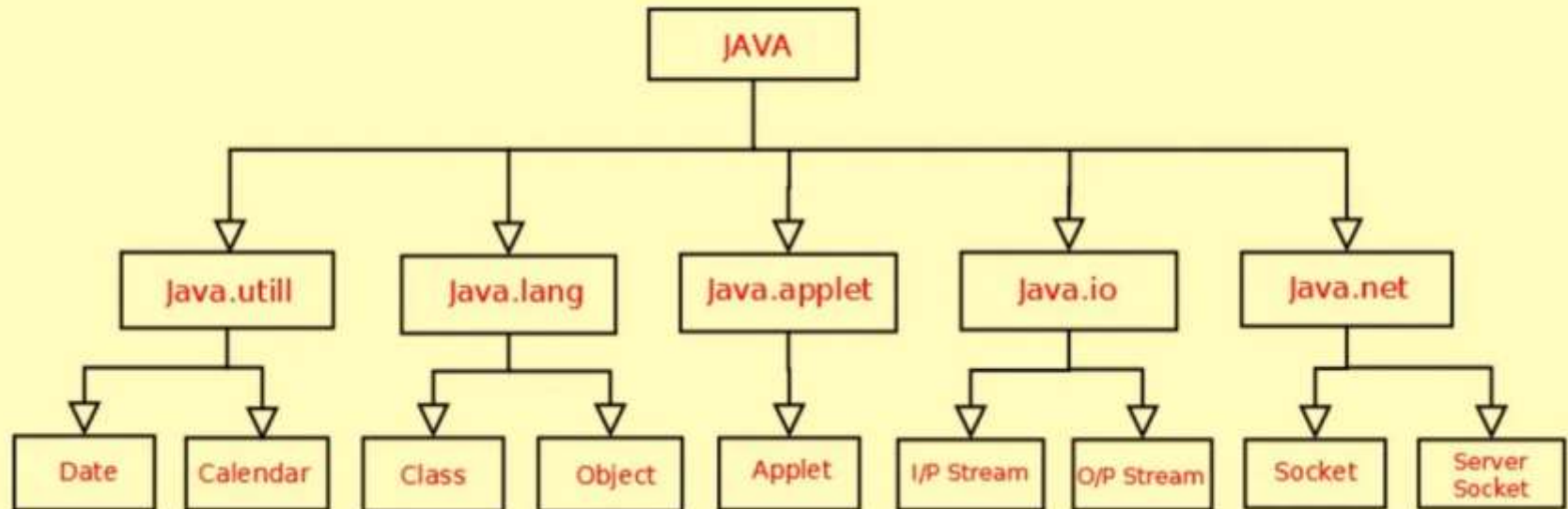


Figure 2.21 Package Organization

## Package Declaration Example

- `package com.parul.oopj;` declares that Student belongs to that package namespace.
- `import java.util.Date;` makes the Date class available without fully qualified names.
- Public classes in a package must reside in a matching folder path on disk.
- Compile from the parent directory using `javac -d .` to place .class files correctly.
- Run with the fully qualified name: `java com.parul.oopj.Student.`
- Packages are essential for reusable libraries and enterprise Java project structure.

## Figure 2.22 Package Example

### PackageExample.java

```
// File: com/parul/oopj/Student.java
package com.parul.oopj;

import java.util.Date;

public class Student {
    private String name;

    public Student(String name) {
        this.name = name;
    }
}

// Compile: javac -d . Student.java
// Run: java com.parul.oopj.Student
```

Figure 2.22 Package Example



**Parul<sup>®</sup>**  
University

**NAAC**  
GRADE **A++**



<https://paruluniversity.ac.in/>

